

Sentencias SQL

Lenguaje SQL

SQL es el lenguaje estándar para trabajar con bases de datos relacionales. Permite definir la estructura de los datos, manipularlos, controlar quién accede a ellos y administrar las transacciones. Sus sentencias se clasifican en cuatro grupos.

Grupo	Nombre	Sentencias
DDL	Definición de datos	CREATE, ALTER, DROP, TRUNCATE
DML	Manipulación de datos	SELECT, INSERT, UPDATE, DELETE
DCL	Control de datos	GRANT, REVOKE
TCL	Control de transacciones	BEGIN, COMMIT, ROLLBACK, SAVEPOINT

Esquema de ejemplo

Tabla	Llave primaria	Atributos	Llave foránea
carrera	id_car	nombre	ninguna
alumno	num_cta	nombre, promedio	id_car hacia carrera
materia	id_mat	nombre, creditos	ninguna
inscripcion	num_cta e id_mat	calificacion	hacia alumno y materia

La tabla inscripcion resuelve la relación muchos a muchos que existe entre alumno y materia.

DDL. Lenguaje de definición de datos

```
CREATE DATABASE escuela;
```

```
CREATE TABLE carrera (  
  id_car SERIAL PRIMARY KEY,  
  nombre VARCHAR(60) NOT NULL UNIQUE  
);
```

```
CREATE TABLE alumno (  
  num_cta INTEGER PRIMARY KEY,  
  nombre VARCHAR(80) NOT NULL,  
  promedio NUMERIC(4,2) CHECK (promedio BETWEEN 0 AND 10),  
  id_car INTEGER REFERENCES carrera(id_car)  
);
```

```
CREATE TABLE materia (  
  id_mat SERIAL PRIMARY KEY,  
  nombre VARCHAR(80) NOT NULL,  
  credits INTEGER DEFAULT 8  
);
```

```
CREATE TABLE inscripcion (  
  num_cta INTEGER REFERENCES alumno(num_cta),  
  id_mat INTEGER REFERENCES materia(id_mat),  
  calificacion NUMERIC(4,2),  
  PRIMARY KEY (num_cta, id_mat)  
);
```

```
ALTER TABLE alumno ADD COLUMN correo VARCHAR(100);
```

```
ALTER TABLE alumno ALTER COLUMN nombre TYPE VARCHAR(120);
```

```
ALTER TABLE alumno DROP COLUMN correo;
```

```
ALTER TABLE alumno RENAME TO estudiante;
```

```
TRUNCATE TABLE inscripcion;
```

```
DROP TABLE inscripcion;
```

DML. Lenguaje de manipulación de datos

```
INSERT INTO carrera (nombre) VALUES ('Computación');
```

```
INSERT INTO alumno (num_cita, nombre, promedio, id_car)  
VALUES (320001, 'Andres Basilio', 9.20, 1),  
       (320002, 'Adriana Garzon', 8.75, 1),  
       (320003, 'Jose Ferrer', 7.40, 2);
```

```
UPDATE alumno SET promedio = 9.50 WHERE num_cita = 320001;
```

```
UPDATE alumno SET promedio = promedio + 0.25 WHERE id_car = 1;
```

```
DELETE FROM alumno WHERE promedio < 6;
```

Consulta básica

```
SELECT * FROM alumno;
```

```
SELECT nombre, promedio FROM alumno;
```

```
SELECT nombre AS estudiante, promedio AS calificacion FROM alumno;
```

```
SELECT DISTINCT id_car FROM alumno;
```

Orden de las cláusulas del SELECT

SELECT columnas que se van a mostrar

FROM tabla de origen

JOIN otra tabla ON condición de enlace

WHERE condición que filtra renglones

GROUP BY columnas de agrupación

HAVING condición que filtra grupos

ORDER BY columnas de ordenamiento

LIMIT número máximo de renglones

Filtrado con WHERE

```
SELECT nombre FROM alumno WHERE promedio >= 9;
```

```
SELECT nombre FROM alumno WHERE id_car = 1 AND promedio > 8;
```

```
SELECT nombre FROM alumno WHERE promedio BETWEEN 8 AND 10;
```

```
SELECT nombre FROM alumno WHERE id_car IN (1, 2);
```

```
SELECT nombre FROM alumno WHERE nombre LIKE 'A%';
```

```
SELECT nombre FROM alumno WHERE promedio IS NULL;
```

Ordenamiento y límite

```
SELECT nombre, promedio FROM alumno ORDER BY promedio DESC;
```

```
SELECT nombre, promedio FROM alumno ORDER BY promedio DESC LIMIT 3;
```

Funciones de agregación

```
SELECT COUNT(*) FROM alumno;
```

```
SELECT AVG(promedio) FROM alumno;
```

```
SELECT MAX(promedio), MIN(promedio) FROM alumno;
```

```
SELECT SUM(creditos) FROM materia;
```

Agrupación

```
SELECT id_car, COUNT(*) AS total  
FROM alumno  
GROUP BY id_car;
```

```
SELECT id_car, AVG(promedio) AS promedio_carrera  
FROM alumno  
GROUP BY id_car  
HAVING AVG(promedio) > 8;
```

La cláusula WHERE filtra renglones antes de agrupar y la cláusula HAVING filtra grupos después de agrupar.

Combinaciones

Tipo de JOIN	Resultado que devuelve
INNER JOIN	Solo los renglones que coinciden en las dos tablas
LEFT JOIN	Todos los de la tabla izquierda más los que coinciden
RIGHT JOIN	Todos los de la tabla derecha más los que coinciden
FULL JOIN	Todos los renglones de ambas tablas
CROSS JOIN	El producto cartesiano de las dos tablas

```
SELECT a.nombre, c.nombre AS carrera
FROM alumno a
INNER JOIN carrera c ON a.id_car = c.id_car;
```

```
SELECT a.nombre, i.calificacion
FROM alumno a
LEFT JOIN inscripcion i ON a.num_cta = i.num_cta;
```

```
SELECT a.nombre, m.nombre AS materia, i.calificacion
FROM alumno a
JOIN inscripcion i ON a.num_cta = i.num_cta
JOIN materia m ON i.id_mat = m.id_mat
ORDER BY a.nombre;
```

Subconsultas

```
SELECT nombre FROM alumno
WHERE promedio > (SELECT AVG(promedio) FROM alumno);
```

```
SELECT nombre FROM alumno
WHERE num_cta IN (SELECT num_cta FROM inscripcion WHERE calificacion = 10);
```

```
SELECT nombre FROM materia m
WHERE EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_mat = m.id_mat);
```

Operadores de conjunto

```
SELECT nombre FROM alumno
```

```
UNION
```

```
SELECT nombre FROM materia;
```

```
SELECT id_mat FROM materia
```

```
EXCEPT
```

```
SELECT id_mat FROM inscripcion;
```

Vistas

```
CREATE VIEW alumnos_destacados AS
```

```
SELECT nombre, promedio FROM alumno WHERE promedio >= 9;
```

```
SELECT * FROM alumnos_destacados;
```

```
DROP VIEW alumnos_destacados;
```

Indices

```
CREATE INDEX idx_alumno_nombre ON alumno (nombre);
```

```
CREATE UNIQUE INDEX idx_materia_nombre ON materia (nombre);
```

```
DROP INDEX idx_alumno_nombre;
```

DCL. Lenguaje de control de datos

```
GRANT SELECT ON alumno TO consulta;
```

```
GRANT SELECT, INSERT, UPDATE ON alumno TO capturista;
```

```
REVOKE INSERT ON alumno FROM capturista;
```

TCL. Control de transacciones

BEGIN;

UPDATE alumno SET promedio = 9.80 WHERE num_cta = 320001;

INSERT INTO inscripcion VALUES (320001, 1, 9.80);

COMMIT;

BEGIN;

DELETE FROM alumno WHERE num_cta = 320003;

ROLLBACK;

BEGIN;

UPDATE alumno SET promedio = 8.00 WHERE num_cta = 320002;

SAVEPOINT punto1;

UPDATE alumno SET promedio = 0 WHERE num_cta = 320002;

ROLLBACK TO punto1;

COMMIT;

Conclusión

Las sentencias SQL cubren todo el ciclo de trabajo con una base de datos. El DDL construye la estructura y define las restricciones que protegen la integridad. El DML carga y consulta la información, y es donde se concentra el mayor uso diario mediante la sentencia SELECT con sus combinaciones y agrupaciones. El DCL asigna los permisos y el TCL asegura que un conjunto de operaciones se aplique de forma completa o no se aplique en absoluto. Dominar estos cuatro grupos permite pasar del diseño teórico del modelo relacional a una base de datos funcionando.